VLSI II Final Project Report

Extension of Croc SoC with DMA Engine and SPI Peripheral

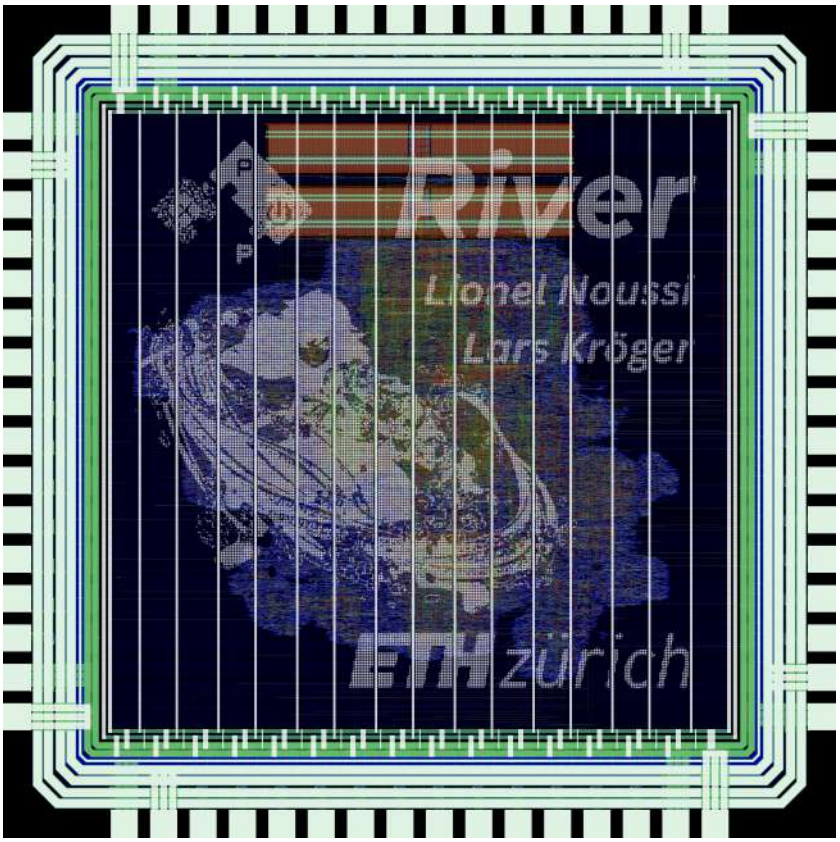
Chip Name: ***RIVER***

Lionel Noussi Lars Kröger

ETH Zürich

July 31, 2025

Abstract. This report documents the design, integration, and physical implementation of a custom System-on-Chip based on the Croc SoC, developed as part of the VLSI II course at ETH Zürich. We extended the baseline design by integrating a general-purpose Direct Memory Access (DMA) engine and a Serial Peripheral Interface (SPI) controller to enhance I/O capabilities and support more realistic embedded workloads. The report covers RTL modifications, verification methodology, backend implementation, and encountered challenges, culminating in a fully placed-and-routed GDSII layout.



Contents

1 Introduction **3**

2 Design **3**

2.1 Architecture Overview 3

2.2 SPI Module Description 4

2.2.1 State Machine 4

2.3 DMA Module Description 5

2.3.1 Block Diagram 5

2.3.2 State Machines 6

2.3.3 Registers & Programming 7

2.4 Software Library 8

2.4.1 Compilation Flow 9

3 Backend Implementation **9**

4 Testing and Verification **11**

4.1 DMA Functional Verification 11

4.2 SPI Functional Verification 11

4.3 Performance and Results 12

4.3.1 Demo Application 12

5 Conclusion **14**

1 Introduction

The goal of this project was to extend the Croc SoC with features that enable more realistic and demanding embedded applications. To this end, we implemented two key components: a general-purpose Direct Memory Access (DMA) engine and a Serial Peripheral Interface (SPI) controller. While both modules are useful individually, their integration enables efficient pipelined data processing with minimal CPU involvement.

The DMA engine facilitates autonomous data transfers between memory and peripherals without CPU involvement. This supports two important use cases: (1) streaming input data from peripherals while the CPU concurrently processes earlier data, and (2) overlapping output streaming with computation of the next frame or result. These capabilities significantly improve system throughput. In addition, the DMA outperforms software-based memory copy loops by avoiding CPU overhead.

The SPI peripheral provides a high-speed, full-duplex communication interface to external devices. In contrast to UART, SPI uses a dedicated clock line, allowing for higher data rates and reduced latency. When combined with the DMA engine, SPI enables sustained high-throughput I/O, critical for real-time applications such as digital signal processing, data logging, or streamed computation.

2 Design

2.1 Architecture Overview

Figure 1 shows the updated SoC architecture. The SPI controller was connected through the existing peripheral demultiplexer, requiring only minor integration effort. In contrast, the DMA engine was attached directly to the system crossbar and introduced three new OBI interfaces.

The DMA module includes one master interface, used by the processor to configure it, and two subordinate interfaces for autonomous memory access: one for reading from a source address, and one for writing to a destination. This decoupling enables pipelined memory transactions and simplifies internal control logic.

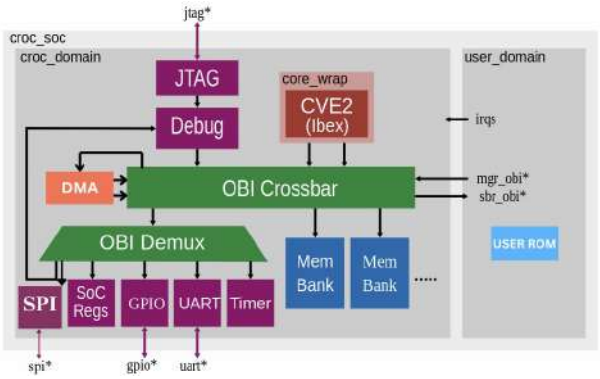


Figure 1: Block diagram of the modified chip

Start Address	Stop Address	Description
0x0000_0000	0x0004_0000	Debug module (JTAG)
0x0300_0000	0x0300_1000	SoC control/info registers
0x0300_2000	0x0300_3000	UART peripheral
0x0300_5000	0x0300_6000	GPIO peripheral
0x0300_A000	0x0300_B000	Timer peripheral
0x0300_C000	0x0300_D000	SPI peripheral
0x1000_0000	0x1000_1000	Memory banks (SRAM)
0x2000_0000	0x5000_0000	Passthrough to user domain
0x2000_0000	0x2000_1000	String-formatted User ROM
0x5000_0000	0x5000_1000	DMA

Table 1: SoC Memory Map

2.2 SPI Module Description

We implemented a general-purpose SPI engine that supports all four SPI modes and allows for runtime configuration of the clock frequency. Its compliance with the strict SPI protocol and support for high data rates make it suitable for a wide range of devices, from simple sensors to memory chips and displays.

2.2.1 State Machine

Table 2: SPI Register Map (relative to base address)

Offset (Hex)	Description
`0x00`	Control register for length or continous mode, triggers activation
`0x04`	Number of bytes transferred during the ongoing transaction
`0x08`	Write register for TX FIFO (pushes one byte into TX buffer)
`0x0C`	Read register for RX FIFO (pops one byte from RX buffer)
`0x18`	Transaction length in bytes
`0x1C`	FIFO buffer status (TX and RX: full, almost full, empty, almost empty)
`0x20`	SPI mode configuration (CPOL and CPHA bits)
`0x24`	Busy flag (1 = transfer ongoing, 0 = idle)
`0x28`	Clock divider to set SPI clock frequency

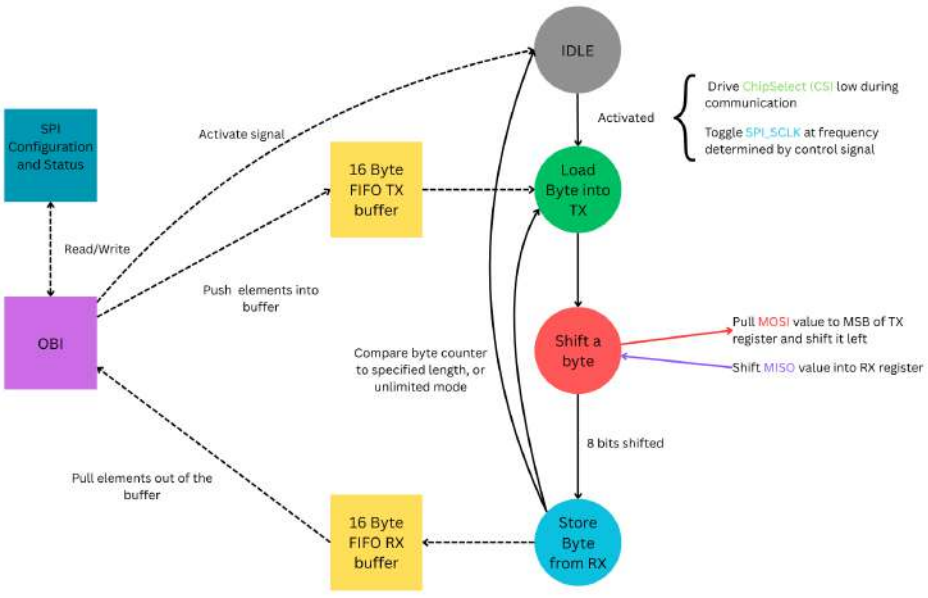


Figure 2: FSM for the SPI module.

The simplified FSM of the SPI Master module is shown above. The SPI mode, the `SPI_CLK` divider, and the transaction length (or a continuous transfer mode) are configured via control registers accessible through the OBI interface. A start flag triggers the beginning of the transfer.

Two 16-byte FIFO buffers are implemented for full-duplex communication: one for transmission (TX) and one for reception (RX), each connected to an 8-bit data register. Write or read accesses to these registers via the OBI automatically push data into or pull data from the buffer.

During the `LOAD` state, the oldest byte from the `TX FIFO` is loaded into the shift register, which is then transmitted over the `MOSI` line during `SHIFT`. Simultaneously, the SPI master samples incoming bits from the `MISO` line into the RX register. Each byte transfer takes 8 cycles of the downsampled `SPI_CLK`, which is continuously generated throughout the transaction. Our implementation supports all SPI modes, which differ in clock polarity (CPOL) and clock phase (CPHA), both of these bits can be set also via a control register, with the default being the most widely used `MODE 0`.

The module supports clock frequencies from 1.25kHz up to 26MHz (with a system clock of 80MHz), enabling data rates up to 26Mbps—roughly 217× faster than UART with at 115kbps. Transfers continue for the specified number of bytes or indefinitely in continuous mode, until interrupted by the core. The core can read and write data from the FIFOs at any time, with two OBI-accessible status registers providing FIFO occupancy (full, almost full, empty, almost empty) and the number of bytes transmitted. Combined with a DMA that monitors buffer status and responds accordingly, this architecture supports efficient high-throughput streaming of large data volumes.

2.3 DMA Module Description

The DMA was designed as a general-purpose data mover, supporting both memory-to-memory and peripheral transfers through a single flexible architecture. This avoids the need for multiple specialized engines (e.g., UART-DMA, SPI-DMA), and reduces system complexity. Configuration is done via a small set of memory-mapped registers, requiring only four writes to initiate a transfer.

2.3.1 Block Diagram

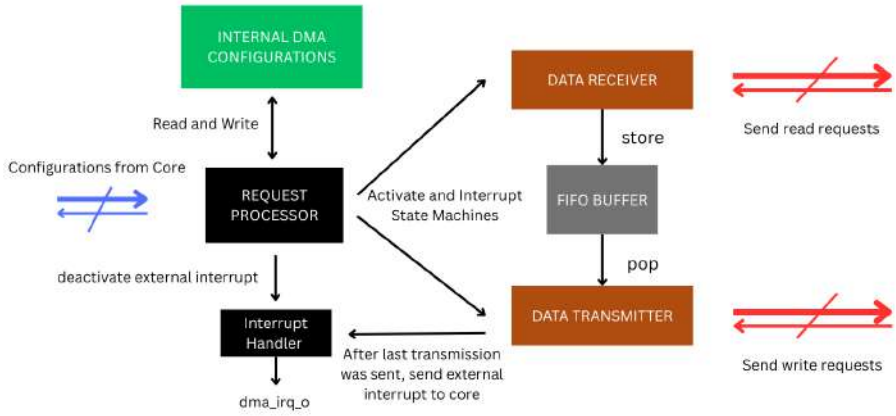


Figure 3: Block diagram of the DMA module.

Figure 3 shows the internal structure of the DMA. The module includes one OBI master port for configuration and two OBI subordinate ports for data movement. This separation allows reads and writes to proceed in parallel, improving throughput and simplifying internal control logic. The price is a slight increase in crossbar size.

The request processor manages access to the DMA's internal configuration registers and controls the overall operation. After configuration, it triggers the receiver and transmitter state machines, which then operate autonomously. The receiver sends data to a First-In-First-Out (FIFO) buffer, and the transmitter pops elements from it. A minimal interrupt handler monitors their completion and interrupts the processor once the transmitter is done. To acknowledge and clear the interrupt, the CPU must write to a specific register. Optionally, the core can also stop the DMA in the middle of a transaction by writing to the same register.

2.3.2 State Machines

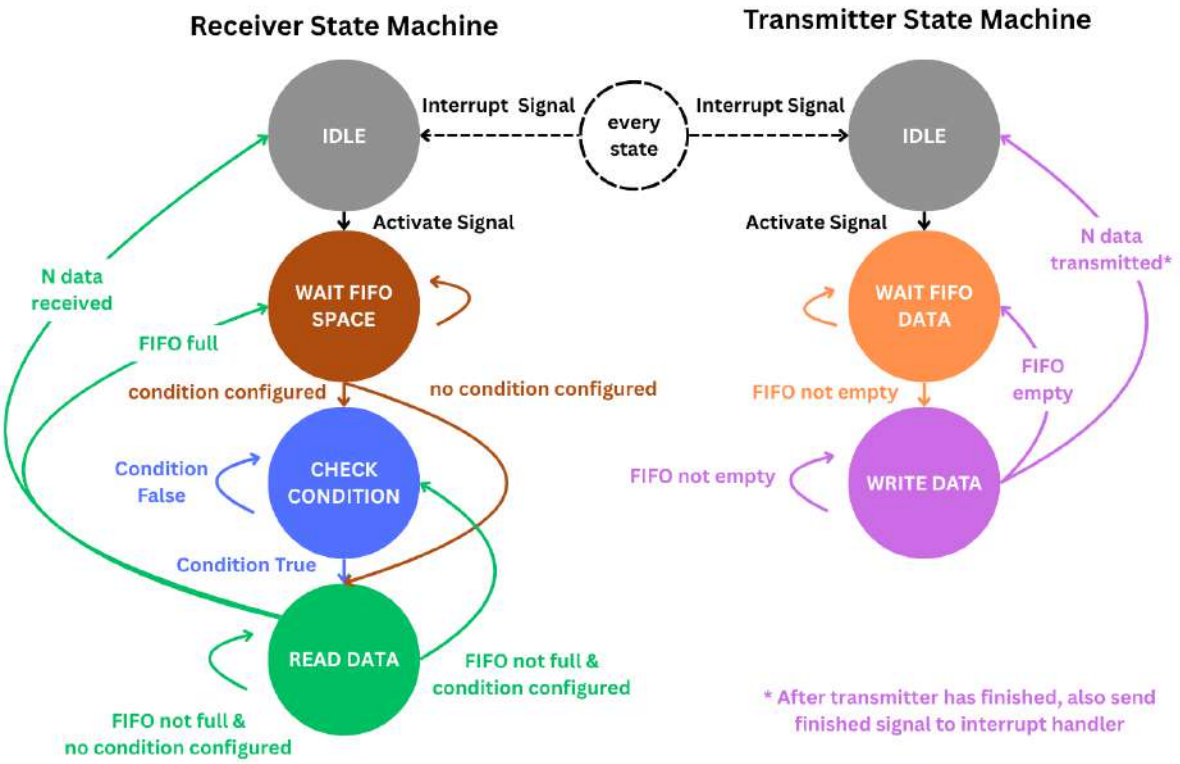


Figure 4: Simplified state machines of the DMA receiver and transmitter.

Figure 4 shows a slightly simplified versions of the data receiver and transmitter state machines used in the DMA module. When the CPU instructs the request processor to start a transfer, it asserts the `activate` signal to both state machines. The receiver begins reading data from the configured source address and writes it into the internal FIFO buffer. The transmitter waits for data in the FIFO, then writes it to the destination address. If the transfer completes or an internal interrupt is triggered, both state machines return to their respective `IDLE` states.

Receiver State Machine The receiver starts in `RECEIVER_IDLE`. If condition checking is enabled, it transitions to `CHECK_CONDITION`, where it reads a byte from the `cond_addr`, applies a boolean mask, and optionally negates the result, depending on the configuration. If the condition evaluates to true, it proceeds to `READ_DATA`; otherwise, it re-evaluates the condition again.

In `READ_DATA`, the receiver issues a read request to the source address and waits for data to return via the OBI interface. Once valid, the data is pushed into the FIFO, a transfer counter is incremented, and—if enabled—the source address is also incremented. If the FIFO becomes full, the receiver transitions to `WAIT_FIFO_SPACE` until space becomes available. The process repeats until the configured number of transfers (`num_transfers`) is reached, at which point the receiver returns to `RECEIVER_IDLE`.

Transmitter State Machine The transmitter starts in `TRANSMITTER_IDLE` and moves to `WAIT_FIFO_DATA`, where it waits for the FIFO to contain data. Once available, it transitions to `WRITE_DATA`, pops a word from the FIFO, and writes it to the configured destination address via the OBI interface. It then increments an internal counter and—if enabled—the destination address.

If more data is available in the FIFO, the transmitter remains in `WRITE_DATA`; otherwise, it returns to `WAIT_FIFO_DATA`. When the counter reaches `num_transfers`, the transmitter transitions back to `TRANSMITTER_IDLE`. If interrupts are enabled, it also signals the DMA interrupt handler, which then asserts `dma_irq_o` to the processor.

Note: For clarity, the state diagram omits handshake-specific intermediate states. Both `CHECK_CONDITION` and `READ_DATA` are implemented as two RTL states: one to issue the read request, and one to wait for a valid response. Additionally, the receiver transitions directly to a data transfer state upon activation, skipping the initial FIFO check since the FIFO is known to be empty.

2.3.3 Registers & Programming

Table 3 lists the DMA register interface, including configuration and control registers, relative to the module's base address.

Offset (Hex)	Register Description
0x00	Source base address
0x04	Destination base address
0x08	Control register (transfer parameters and flags)
0x0C	Condition register (optional transfer trigger configuration)
0x10	Interrupt register (interrupt DMA or stop external interrupt)
0x14	Activate register (start transfer, write-only)
0x18	Status register (read-only)

Table 3: DMA Register Map (relative to base address)

The 32-bit control and condition registers match the OBI signal width, allowing single-write configuration. A more detailed break-down of the registers can be seen below.

Control Register Layout

src_offset	dst_offset	num_transfers	irq?	inc_src	inc_dst	size	activate
31:24	23:16	15:5	4	3	2	1	0

Condition Register Layout

cond_addr_offset	bitmask	cond_base	negate	enable
31:24	23:16	2	1	0

`num_transfers` is 11 bits wide, supporting up to 2KB per transfer—sufficient for the current 4KB memory, but should be extended in future designs. The `size` bit selects WORD=0 or BYTE=1 transfers. Setting `activate` also immediately starts the transfer. The Condition Register enables optional checks before each transfer. `cond_base` selects whether the condition uses the source or destination address as a base for `cond_addr_offset`.

2.4 Software Library

To simplify DMA and SPI usage, we provide a set of high-level software functions that abstract common transfer operations (see Figure 5).

```
void spi_init(uint8_t spi_mode, uint16_t clk_divider);
void spi_empty_rx();
void spi_read(uint8_t* destination_array, uint16_t num_bytes);
void spi_write(uint8_t* source_array, uint16_t num_bytes);
void spi_read_write(uint8_t* data_tx, uint8_t* data_rx,  uint16_t num_bytes);

void uart_read_dma(uint8_t* target_buffer, uint16_t num_bytes);
void uart_write_dma(uint8_t* source_buffer, uint16_t num_bytes);
void spi_read_dma(uint8_t* target_buffer, uint16_t num_bytes);
void spi_write_dma(uint8_t* source_buffer, uint16_t num_bytes);
void* memcpy_dma(void* dst, const void* src, unsigned num_bytes);

int dma_busy();
void enable_dma_irq();
void __attribute__((weak)) dma_irq_handler_user();
```

Figure 5: Core user functions for peripheral setup, DMA-based data transfers, and interrupt support

spi.c The SPI is first initialized with the desired mode and a clock divider. A helper function is available to flush the `RX FIFO` buffer. Basic read, write, and read/write functions continuously monitor the FIFO status and push or pull data until the specified number of bytes has been transferred, in a blocking manner.

dma.c In addition to various helper functions and structs to manually program the DMA, dedicated functions have been created for most common data movements, as listed in Figure 5. Status and control functions, such as `dma_busy` and `interrupt_dma` support the user.

interrupts.c The software library has been extended with full interrupt support, which proved to be a challenging task. The `crt0.S` start-up script now sets the `mtvec` register to an 8-byte aligned trap vector table, enabling per-interrupt-ID service routines. This table invokes functions in `interrupts.c` marked with the `__interrupt__` attribute. For DMA interrupts, the table jumps to a dedicated DMA IRQ handler, which clears the interrupt signal and calls the weak `dma_irq_handler_user` function. This mechanism lets users define their own handler that is triggered once the DMA transfer completes. The same approach can be applied to other or future interrupts.

2.4.1 Compilation Flow

As an additional enhancement, we restructured the Makefile to compile all files in the `lib` directory into a static library. The main application is then linked against this library with garbage collection enabled, ensuring that only used functions are included in the final hex file. This significantly reduced the size of the `.text` section, saving valuable SRAM space.

3 Backend Implementation

The backend flow was implemented using Yosys for synthesis, OpenROAD for physical design, and Calibre DRV for Design Rule Checks (DRC) and Layout-Versus-Schematic (LVS). We initially reimplemented all scripts from scratch, based on course exercises and documentation. Eventually, we selectively integrated parts of the reference flow we found useful. The final design meets the 80 MHz timing target and passes all hold, DRC, and LVS checks.

Antenna Violations

Antenna violations were the most challenging part of the backend flow. Even before the updated OpenROAD version was released, we spent significant time debugging over 400 violations. After the update, around 30 remained, and the automated repairs degraded timing and introduced many new fanout violations.

We discovered that the default flow—calling `repair_antennas` before `detailed_routing`—was problematic. Attempting to run `detailed_routing`, then `repair_antennas`, and then rerun `detailed_routing` caused OpenROAD to crash with an error about bondpads lying outside the chip area.

We resolved this by adjusting the floorplan: increasing the total OpenROAD floorplan araea, but also increasing the chip area margin, places the bondpads fully inside the OpenROAD-defined area. With this change, `repair_antennas` could be run **after** detailed routing without issues. Only a single iteration was required to resolve nearly all violations—unlike the five suggested in the reference flow—minimizing negative impact on timing. The final DRC reported just three antenna violations, which all resolved by manually fixing just one.

Floorplan Offset

While the floorplan changes fixed the antenna violations, they introduced a new issue: bondpads were offset by 70 μm up and right, violating the required coordinates. We corrected this by modifying the `def2gds` script to shift all cells by 70 μm down and left, restoring proper alignment.

Timing

Our design met the 80 MHz frequency target. Aside from minor degradation caused by antenna repairs (which was later resolved), we encountered no significant timing issues. Both the DMA and SPI modules were implemented efficiently and remained decoupled from the core, avoiding long combinational paths. Increasing the crossbar size with two additional inputs and one output also had no measurable impact.

We estimated final timing by calling `estimate_parasitics -global_routing` after the first detailed routing, which reported a worst negative setup slack (WNS) of 0.07 ns and a final hold WNS of 0.1 ns.

This estimate appeared most reliable, as running `estimate_parasitics -global_routing` after `repair_antennas -ratio_margin 30 -iterations 1` produced warnings about missing routes and showed improved setup but degraded hold slack. We suspect this is due to global routing data being consumed or altered by `repair_antennas`, leading to inaccurate parasitic estimation. Since the reported slack was still positive, we concluded that inserting a few diodes during antenna repair would not significantly impact final timing. Trying to read-back the spef file in OpenROAD also produced extremely pessimistic results, and we were advised by teaching assistants to not use this method.

Macro Placement and PDN Integration

We encountered minor macro alignment issues: one SRAM macro triggered a grid alignment warning, which we resolved by manually adjusting placement. We also observed PDN conflicts near macros, where vertical core power stripes intersected the SRAM power ring. Increasing the macro halo resolved these violations and removed all related warnings.

Clock Tree Synthesis and Fanout Tradeoffs

OpenROAD's clock tree synthesis (CTS) offers limited configurability. We experimented with sink clustering, adjusting levels, balance, and size. With the tuned parameters, all fanout violations on the system clock were eliminated. A single fanout violation of 16 (maximum allowed: 8) remained on one JTAG clock buffer, but it was deemed acceptable as the net stayed within the maximum capacitance limits and did not impact timing. We interpreted the fanout limit primarily as a proxy for skew and capacitive loading, both of which remained under control.

LVS

Initial LVS runs failed due to mismatched power net names. We traced this to pin placement being executed before power connectivity in our script, causing incorrect net extraction. Cross-

checking with the reference flow's order—connecting power nets first—resolved the issue, and LVS passed without further errors.

Final Results

Final signoff showed only the three antenna violations and five minimum metal area violations, all of which we fixed manually. LVS returned no mismatches. All OpenROAD warnings were reviewed and either resolved or justified. No setup or hold violations remain and our system clock runs at 80MHz.

4 Testing and Verification

4.1 DMA Functional Verification

The DMA module was developed and tested incrementally, starting with basic subordinate interface functionality and ending with full system-level integration. In early stages, we focused on writing and reading back configuration registers through the subordinate interface to verify register mapping and access correctness.

Once the configuration path was validated, we implemented the transmitter state machine and tested simple transfers of constant values to memory using a programmed destination address. Afterward, we introduced the FIFO buffer and a preliminary version of the data receiver, which injected dummy data into the FIFO. This allowed us to test the interaction between the receiver and transmitter, including bursty behavior and flow control.

With this infrastructure in place, we completed the implementation of the full data receiver state machine, first without and then with condition checking enabled. Each new component or mode was thoroughly tested in isolation before moving to the next step.

We also added interrupt support to allow the CPU to be notified when transfers completed. While the RTL changes were straightforward, the integration on the software side was more involved as described earlier.

To simulate incoming data, we modified the testbench to send data via UART when it received a trigger character from the core (e.g., 0x01). This enabled testing DMA-to-UART and UART-to-DMA transfers interactively. To test the SPI functionality, the aforementioned testbench modules were used.

In the final stage, we created multiple comprehensive scripts in C, verifying the three main DMA use cases:

1. Streaming data from a peripheral to memory.
2. Streaming data from memory to a peripheral.
3. Copying data from one memory location to another.

4.2 SPI Functional Verification

The SPI testbench implements a general-purpose SPI slave that receives bytes from the master and processes them according to a configurable protocol. In the default configuration, the slave

simply logs received data or writes it to an internal buffer, enabling functional verification of the SPI engine across all four SPI modes and verifying correct behavior on all required clock edges. It also confirms correct operation at the minimum supported clock divider of 3, corresponding to a 26MHz SPI clock at an 80,MHz system clock.

For more advanced testing, the base testbench was extended to simulate an SSD device, where the slave interprets a structured command format: a control byte (read/write), a transfer length, and a 16-bit address split into ADDR_HIGH and ADDR_LOW. In this case, the slave either writes data to or reads data from an internal memory array, auto-incrementing the address during transfers. A second variant emulates a simple video monitor by writing received data to a hex file after a fixed number of bytes.

4.3 Performance and Results

4.3.1 Demo Application

To demonstrate the capabilities of our DMA and SPI modules, we developed a real-time graphics demo that renders a monochrome wave animation on a pixel grid.

The demo can be run with and without DMA. When using DMA, we implement double buffering to compute the next frame while the previous one is being transferred. Because the DMA engine handles the data movement autonomously, the CPU is freed for computation during transmission. This resulted in a speedup of approximately 1.75× for our demo compared to the non-DMA version—approaching the ideal factor of 2× when computation and transfer times are equal.

The performance benefits the SPI offers are particularly clear when comparing it to UART, the only other streaming peripheral available to the core. For our final demo, we configured the SPI clock to a realistic 10MHz, enabling video output at around 30fps. In contrast, UART's data rate is roughly 100× lower, making real-time video output infeasible.

A short video of the demo is available here: https://youtu.be/-UqgZtOXc2w

Memory-to-Memory Copy To demonstrate the DMA's performance advantage, we compared it against an optimized software `memcpy`. While both methods take similar time for small transfers (e.g., 32 bytes), the DMA quickly outperforms the software as the size increases (Figure 6). This speedup stems from the DMA's ability to receive and send data in parallel, whereas the software incurs loop overhead. The initial setup and interrupt handling introduce some fixed DMA overhead, making it less efficient for very short transfers. However, since the CPU remains unblocked during a DMA transfer, it can perform other tasks in parallel, further improving overall system efficiency.

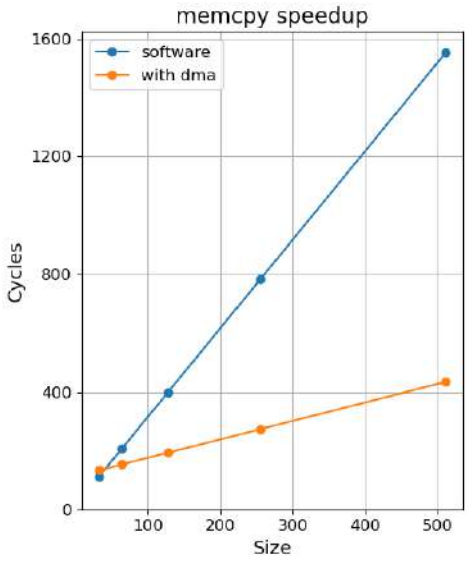


Figure 6: memcpy transfer time comparison with and without DMA, for different array sizes in Bytes.

Other Demo Applications We showcased the DMA's advantages through additional double-buffered demo scripts. One example streams audio data over UART, using the DMA to fill input buffers while a discrete Fourier transform (DFT) runs on the other—enabling simple keyword detection. We also demonstrated support for external hard drive access, a key feature given the limited on-chip memory. All demos can be found in our github repository. Other potential applications include real-time motor control, wearable devices that continuously process SPI sensor data, and automated status logging.

Design Constraints and Observations Through the experiments we made, we observed that to realize actual speedup, the transferred data size must exceed the internal buffers of UART or SPI, which are both only 16 bytes long. Otherwise, pseudo-double-buffering could be approximated by pushing or reading all data in a single block, negating the DMA's advantage. Furthermore, the computation must take longer than it takes the peripheral to fill or empty its buffer; otherwise, post/pre-processing is fast enough without losing any bytes. However, both of these requirements are easily satisfied in real-world applications.

5 Conclusion

In this project, we successfully extended the Croc SoC with a general-purpose DMA engine and an SPI peripheral, enabling parallel computation and data movement for realistic real-time applications. The DMA was designed for flexibility and performance, supporting memory-to-memory transfers, peripheral streaming, and interrupt-driven operation.

We verified our design through systematic simulation, from individual module testing to full end-to-end application scenarios. Our backend flow was rebuilt and customized from the ground up, with special attention to antenna repair, floorplanning, and clock tree synthesis. The final chip runs at 80MHz and passes all DRC and LVS checks.

Finally, our demo shows tangible performance improvements in I/O-bound scenarios—particularly through double-buffered DMA streaming and the high data-rate of the SPI module. These gains validate our design choices and demonstrate that the added hardware not only functions correctly, but also significantly improves system throughput under realistic workloads.

GitHub Repository

To see the source code of our RTL and testbenches, as well as our synthesis, place and route scripts, visit https://github.com/LionelNoussi/croc